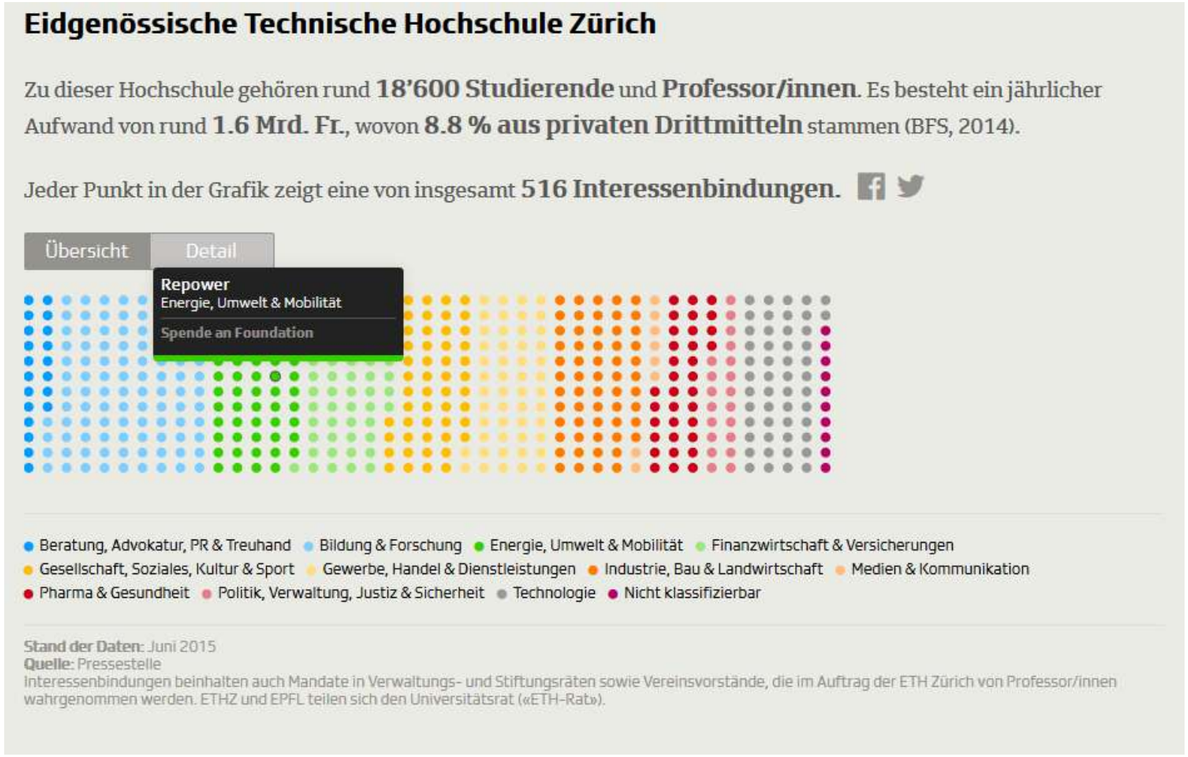
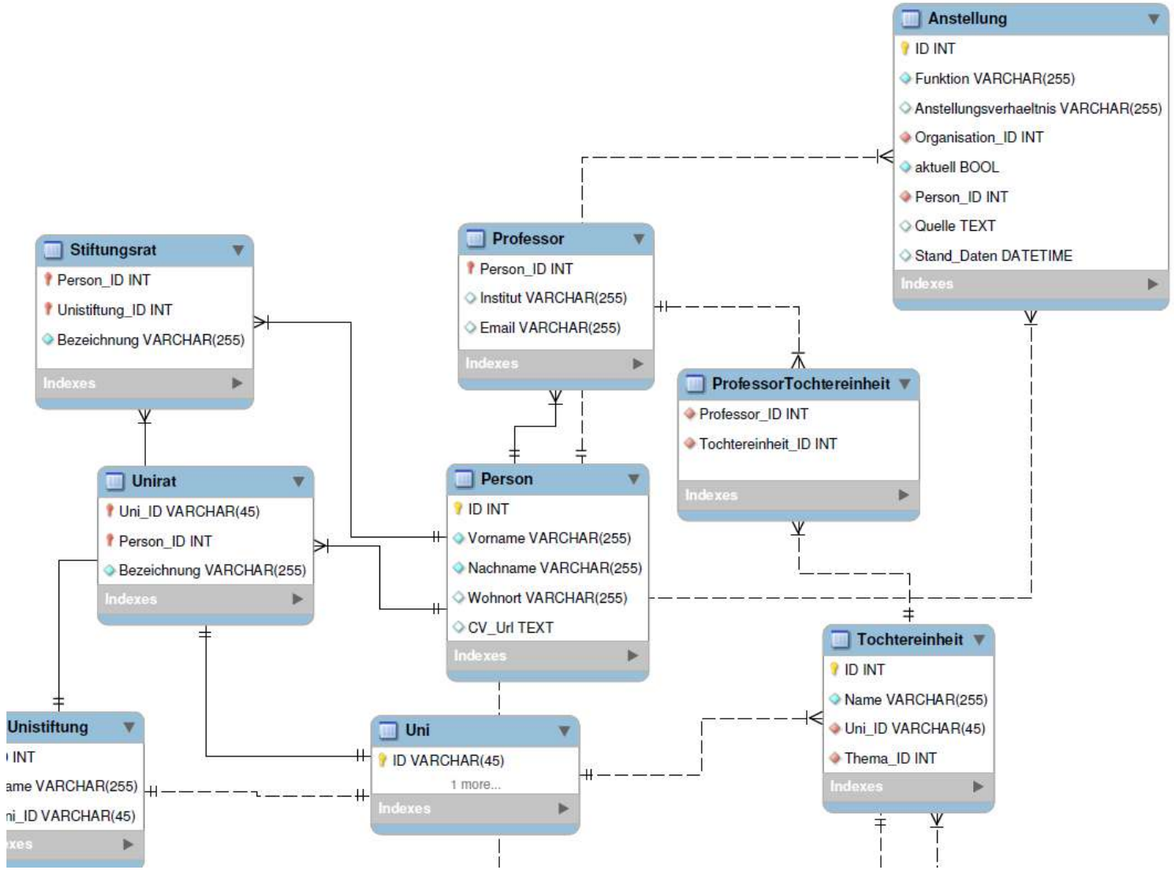
Your first database

INTRODUCTION TO RELATIONAL DATABASES IN SQL

Timo Grossenbacher
Data Journalist

Investigating universities in Switzerland





A relational database:

- real-life *entities* become *tables*
- reduced redundancy
- data integrity by *relationships*

- e.g. `professors`, `universities`, `companies`
- e.g. only one entry in `companies` for the bank "Credit Suisse"
- e.g. a `professor` can work at multiple `universities` and `companies`, a `company` can employ multiple `professors`

Throughout this course you will:

- work with the data I used for my investigation
- create a relational database from scratch
- learn three concepts:
  - *constraints*
  - *keys*
  - *referential integrity*

You'll need: Basic understanding of SQL, as taught in **Introduction to SQL**.

Your first duty: Have a look at the PostgreSQL database

```
SELECT table_schema, table_name
FROM information_schema.tables;
```

```
   table_schema    |              table_name
 ------------------+------------------------------
 pg_catalog        | pg_statistic
 pg_catalog        | pg_type
 pg_catalog        | pg_policy
 pg_catalog        | pg_authid
 pg_catalog        | pg_shadow
 public            | university_professors
 pg_catalog        | pg_settings
...
```

Have a look at the columns of a certain table

```
SELECT table_name, column_name, data_type
FROM information_schema.columns
WHERE table_name = 'pg_config';
```

```
 table_name | column_name | data_type
------------+-------------+-----------
 pg_config  | name        | text
 pg_config  | setting     | text
```

Let's do this.

INTRODUCTION TO RELATIONAL DATABASES IN SQL

Tables: At the core of every database

INTRODUCTION TO RELATIONAL DATABASES IN SQL

Timo Grossenbacher
Data Journalist

datacamp

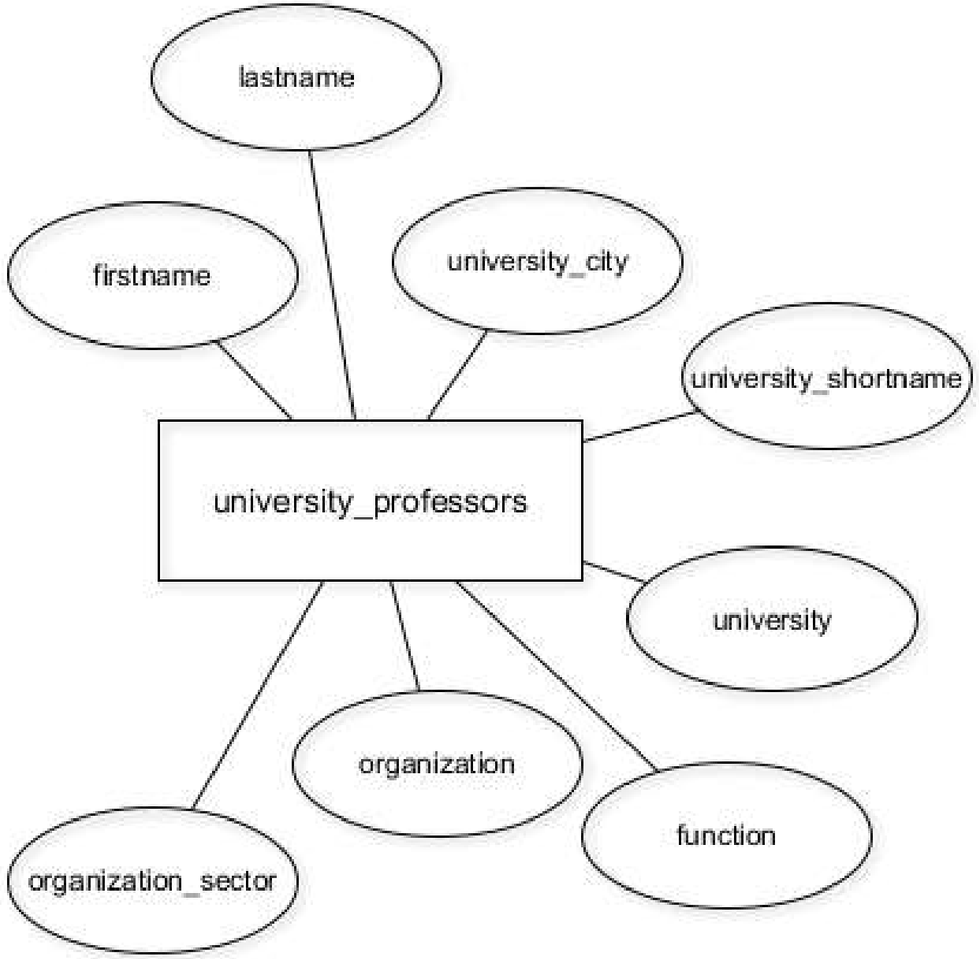
Redundancy in the university_professors table

```
SELECT *
FROM university_professors
LIMIT 3;
```

```
-[ RECORD 1 ]--------+--------------------------------------------------
firstname            | Karl
lastname             | Aberer
university           | ETH Lausanne
university_shortname | EPF
university_city      | Lausanne
function             | Chairman of L3S Advisory Board
organization         | L3S Advisory Board
organization_sector  | Education & research
-[ RECORD 2 ]--------+--------------------------------------------------
firstname            | Karl
lastname             | Aberer
university           | ETH Lausanne
university_shortname | EPF
university_city      | Lausanne
function             | Member Conseil of Zeno-Karl Schindler Foundation
organization         | Zeno-Karl Schindler Foundation
organization_sector  | Education & research
-[ RECORD 3 ]--------+--------------------------------------------------
firstname            | Karl
lastname             | Aberer
(truncated)
function             | Member of Conseil Fondation IDIAP
organization         | Fondation IDIAP
(truncated)
```

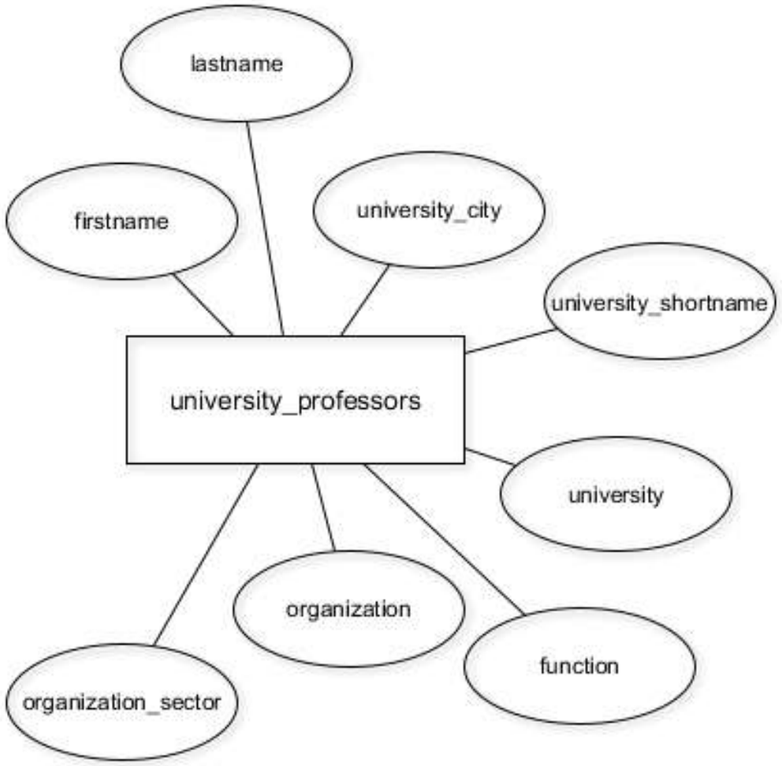
```
-[ RECORD 1 ]--------+-----------------------------------------
firstname            | Karl
lastname             | Aberer
university           | ETH Lausanne
university_shortname | EPF
university_city      | Lausanne
function             | Chairman of L3S Advisory Board
organisation         | L3S Advisory Board
organisation_sector  | Education & research
-[ RECORD 2 ]--------+-----------------------------------------
firstname            | Karl
lastname             | Aberer
university           | ETH Lausanne
university_shortname | EPF
university_city      | Lausanne
function             | Member Conseil of Zeno-Karl Schindler Foundation
organisation         | Zeno-Karl Schindler Foundation
organisation_sector  | Education & research
-[ RECORD 3 ]--------+-----------------------------------------
firstname            | Karl
lastname             | Aberer
(truncated)
function             | Member of Conseil Fondation IDIAP
organisation         | Fondation IDIAP
(truncated)
```

Currently: One "entity type" in the database

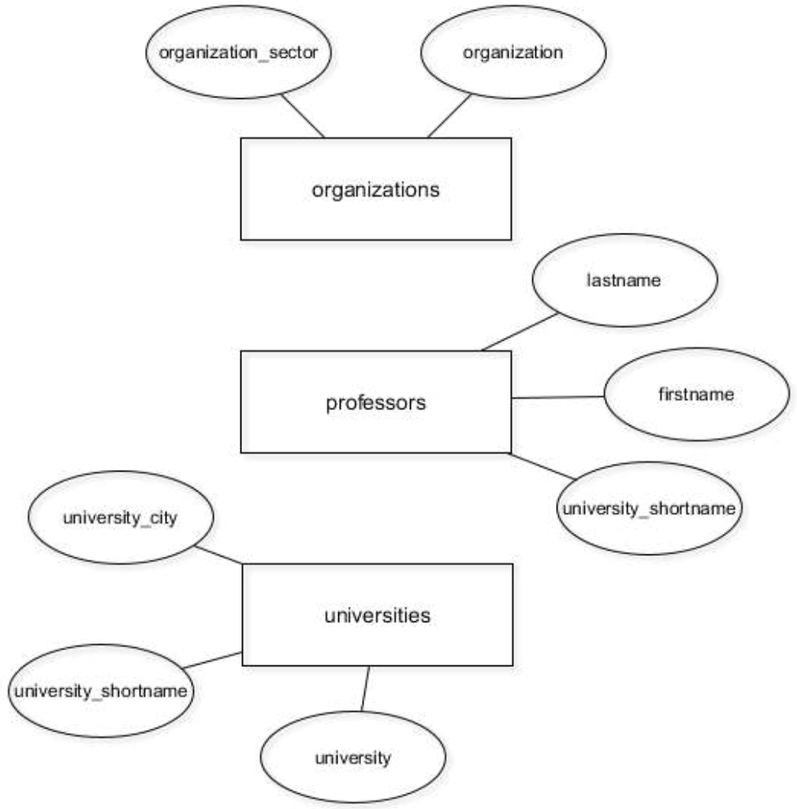


A better database model with three entity types

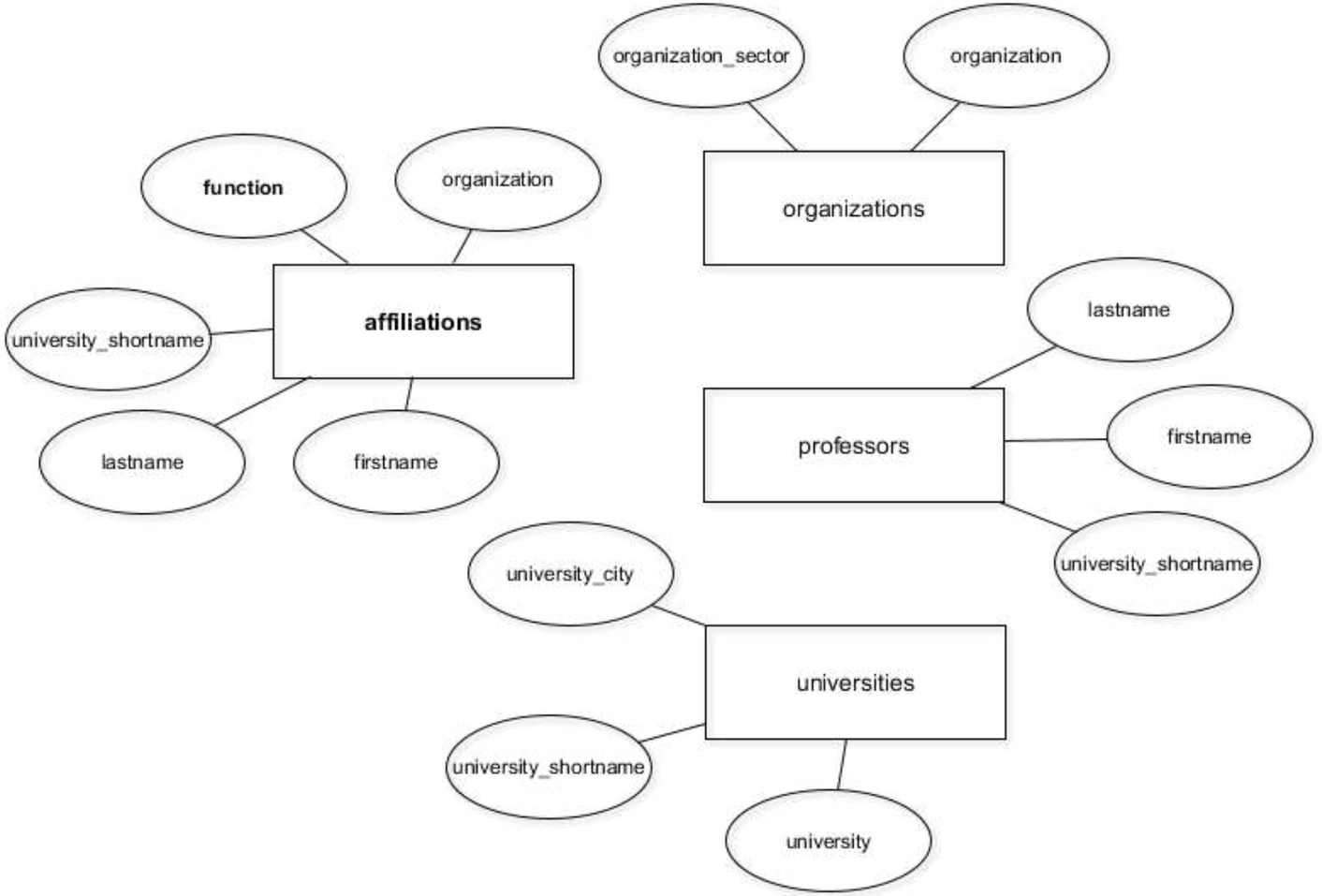
Old:



New:



A better database model with four entity types

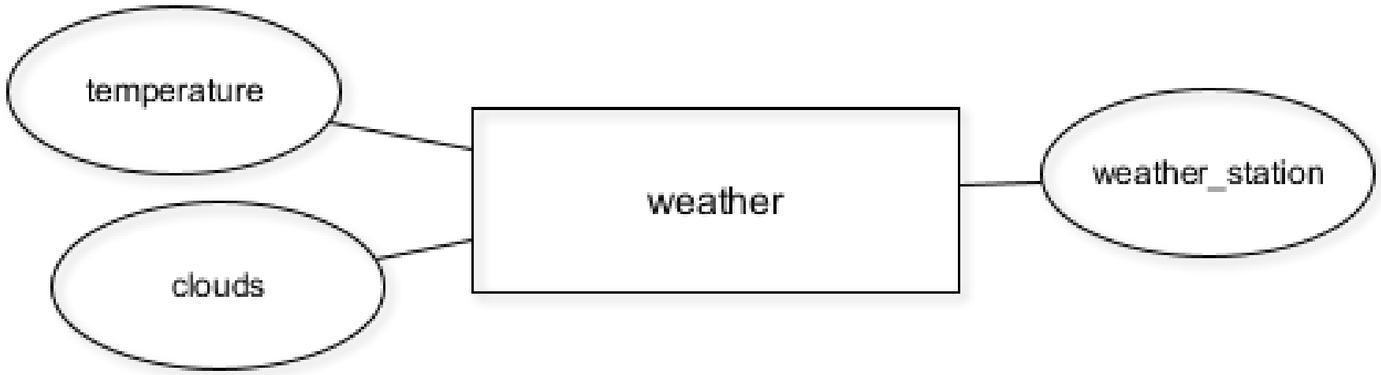


Create new tables with CREATE TABLE

```
CREATE TABLE table_name (
 column_a data_type,
 column_b data_type,
 column_c data_type
);
```

Create new tables with CREATE TABLE

```
CREATE TABLE weather (
 clouds text,
 temperature numeric,
 weather_station char(5)
);
```



Let's practice!

INTRODUCTION TO RELATIONAL DATABASES IN SQL

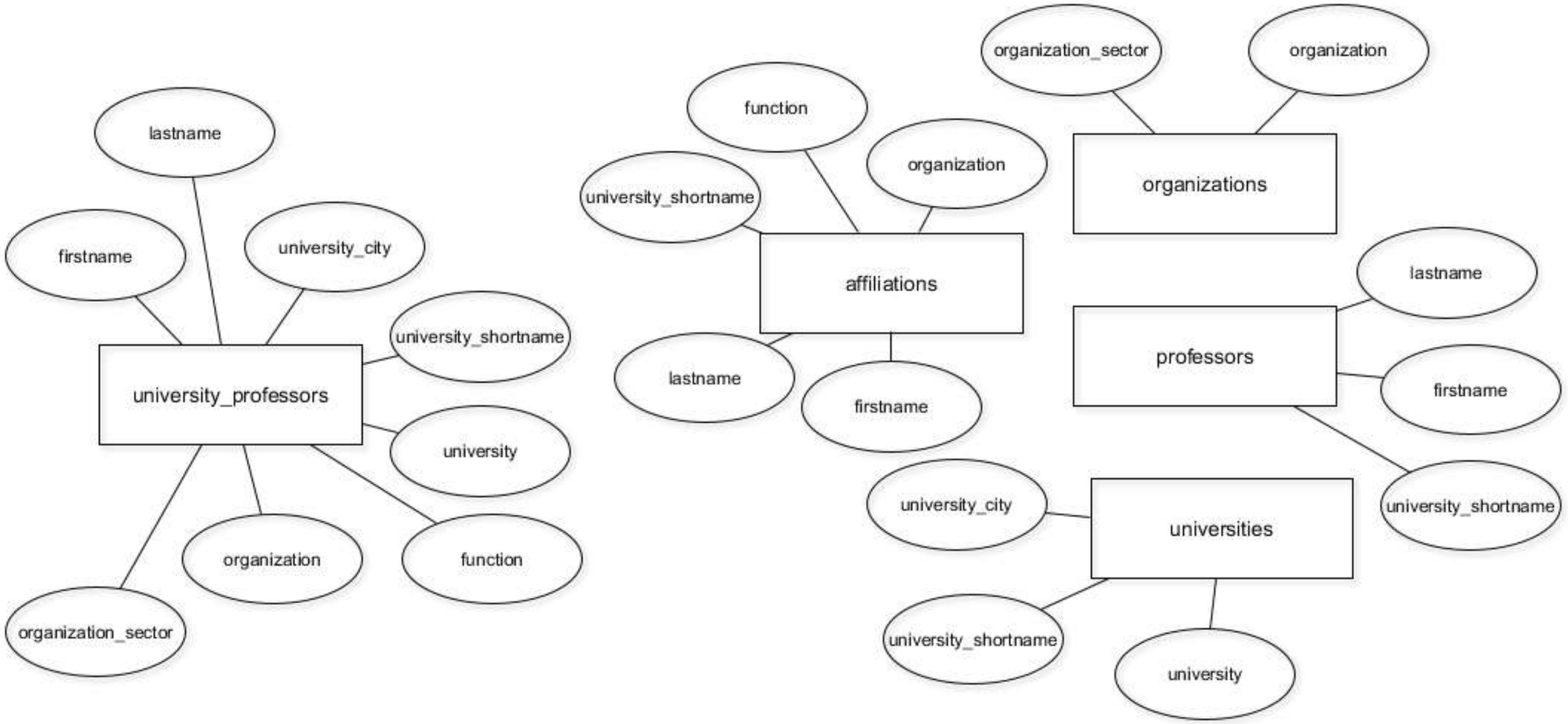
Update your database as the structure changes

INTRODUCTION TO RELATIONAL DATABASES IN SQL

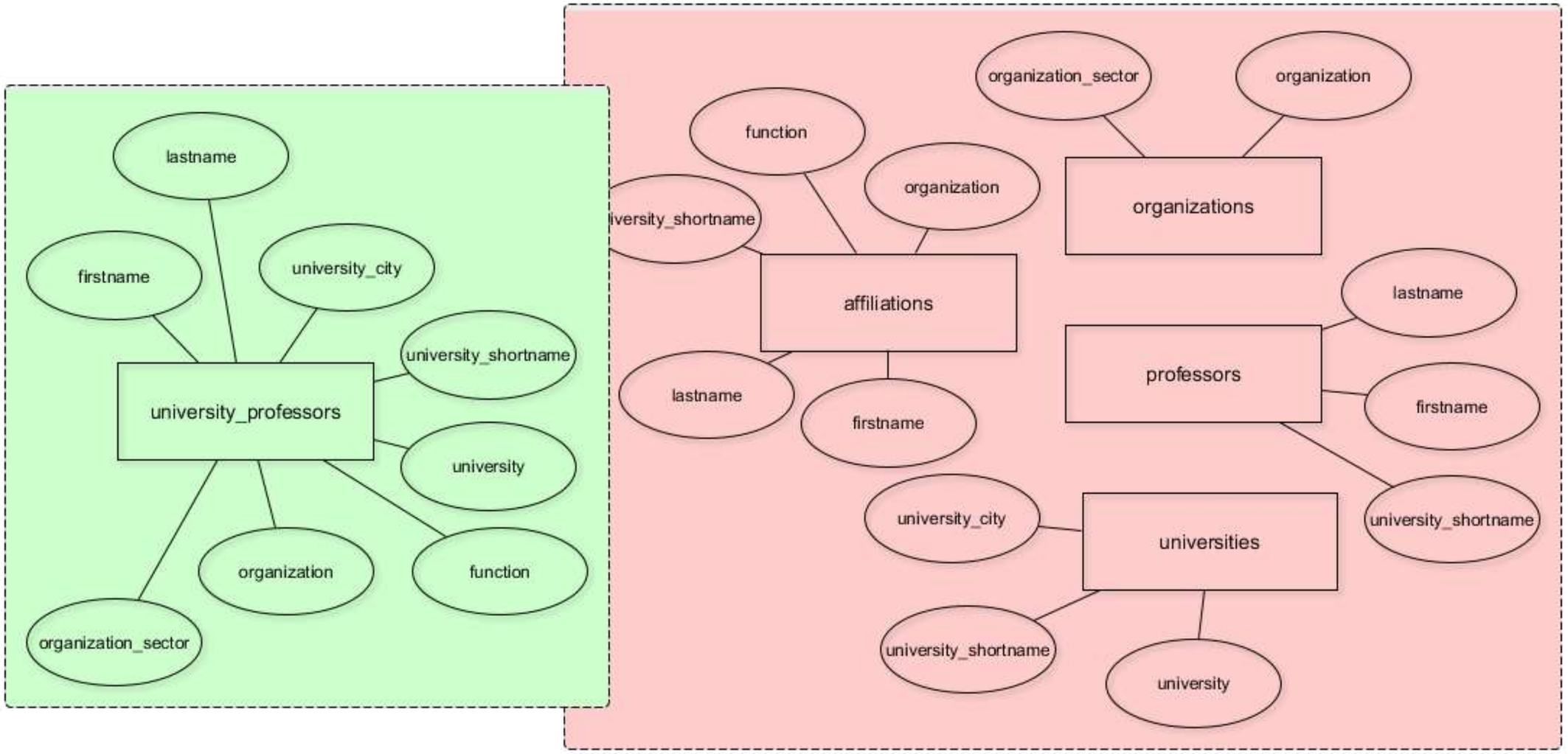
Timo Grossenbacher
Data Journalist

datacamp

The current database model



The current database model



Only store DISTINCT data in the new tables

```
SELECT COUNT(*)
FROM university_professors;
```

```
count
-----
1377
```

```
SELECT COUNT(DISTINCT organization)
FROM university_professors;
```

```
count
-----
1287
```

INSERT DISTINCT records INTO the new tables

```
INSERT INTO organizations
SELECT DISTINCT organization,
    organization_sector
FROM university_professors;
```

```
Output: INSERT 0 1287
```

```
INSERT INTO organizations
SELECT organization,
    organization_sector
FROM university_professors;
```

```
Output: INSERT 0 1377
```

The INSERT INTO statement

```
INSERT INTO table_name (column_a, column_b)
VALUES ("value_a", "value_b");
```

RENAME a COLUMN in affiliations

```
CREATE TABLE affiliations (
 firstname text,
 lastname text,
 university_shortname text,
 function text,
 organisation text
);
```

```
ALTER TABLE table_name
RENAME COLUMN old_name TO new_name;
```

DROP a COLUMN in affiliations

```
CREATE TABLE affiliations (
 firstname text,
 lastname text,
 university_shortname text,
 function text,
 organization text
);
```

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

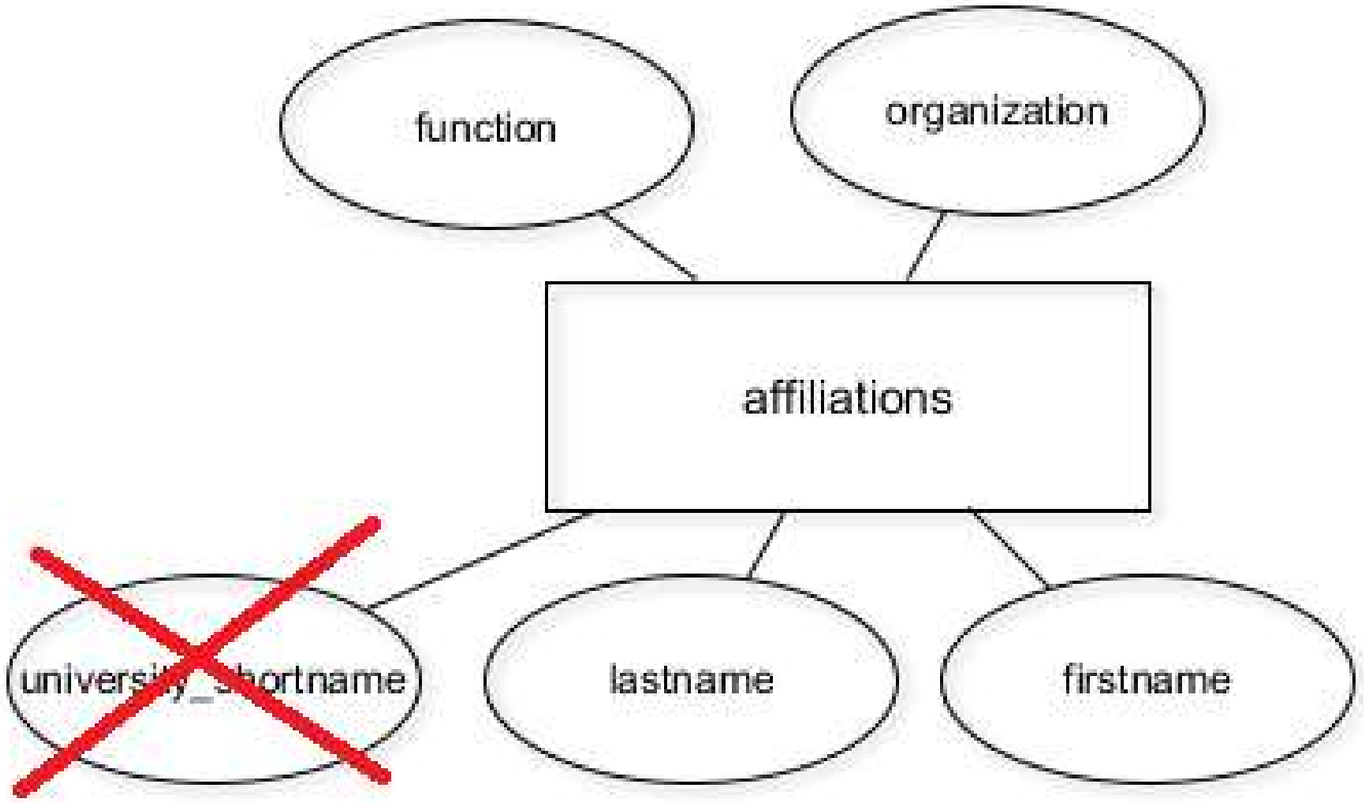
```
SELECT DISTINCT firstname, lastname,
    university_shortname
FROM university_professors
ORDER BY lastname;
```

```
-[ RECORD 1 ]--------+-------------
firstname            | Karl
lastname             | Aberer
university_shortname | EPF
-[ RECORD 2 ]--------+-------------
firstname            | Reza Shokrollah
lastname             | Abhari
university_shortname | ETH
-[ RECORD 3 ]--------+-------------
firstname            | Georges
lastname             | Abou Jaoudé
university_shortname | EPF
(truncated)

(551 records)
```

```
SELECT DISTINCT firstname, lastname
FROM university_professors
ORDER BY lastname;
```

```
-[ RECORD 1 ]----------------------
firstname | Karl
lastname  | Aberer
-[ RECORD 2 ]----------------------
firstname | Reza Shokrollah
lastname  | Abhari
-[ RECORD 3 ]----------------------
firstname | Georges
lastname  | Abou Jaoudé
(truncated)

(551 records)
```

A professor is uniquely identified by firstname, lastname only



Let's get to work!

INTRODUCTION TO RELATIONAL DATABASES IN SQL

Better data quality with constraints

INTRODUCTION TO RELATIONAL DATABASES IN SQL

Timo Grossenbacher
Data Journalist

datacamp

Integrity constraints

1. **Attribute constraints**, e.g. data types on columns (Chapter 2)
2. **Key constraints**, e.g. primary keys (Chapter 3)
3. **Referential integrity constraints**, enforced through foreign keys (Chapter 4)

Why constraints?

- Constraints give the data structure
- Constraints help with consistency, and thus data quality
- Data quality is a business advantage / data science prerequisite
- Enforcing is difficult, but PostgreSQL helps

Data types as attribute constraints

Name	Aliases	Description
`bigint`	`int8`	signed eight-byte integer
`bigserial`	`serial8`	autoincrementing eight-byte integer
`bit [ (n) ]`		fixed-length bit string
`bit varying [ (n) ]`	`varbit [ (n) ]`	variable-length bit string
`boolean`	`bool`	logical Boolean (true/false)
`box`		rectangular box on a plane
`bytea`		binary data ("byte array")
`character [ (n) ]`	`char [ (n) ]`	fixed-length character string
`character varying [ (n) ]`	`varchar [ (n) ]`	variable-length character string
`cidr`		IPv4 or IPv6 network address

From the PostgreSQL documentation.

Dealing with data types (casting)

```
CREATE TABLE weather (
 temperature integer,
 wind_speed text);
SELECT temperature * wind_speed AS wind_chill
FROM weather;
```

```
operator does not exist: integer * text
HINT: No operator matches the given name and argument type(s).
You might need to add explicit type casts.
```

```
SELECT temperature * CAST(wind_speed AS integer) AS wind_chill
FROM weather;
```

Let's practice!

INTRODUCTION TO RELATIONAL DATABASES IN SQL

Working with data types

INTRODUCTION TO RELATIONAL DATABASES IN SQL

Timo Grossenbacher
Data Journalist

datacamp

Working with data types

- Enforced on columns (i.e. attributes)
- Define the so-called "domain" of a column
- Define what operations are possible
- Enforce consistent storage of values

The most common types

- `text` : character strings of any length
- `varchar [ (x) ]` : a maximum of `x` characters
- `char [ (x) ]` : a fixed-length string of `x` characters
- `boolean` : can only take three states, e.g. `TRUE` , `FALSE` and `NULL` (unknown)

From the **PostgreSQL documentation**.

The most common types (cont'd.)

- `date`, `time` and `timestamp`: various formats for date and time calculations
- `numeric`: arbitrary precision numbers, e.g. `3.1457`
- `integer`: whole numbers in the range of `-2147483648` and `+2147483647`

From the **PostgreSQL documentation**.

Specifying types upon table creation

```
CREATE TABLE students (
 ssn integer,
 name varchar(64),
 dob date,
 average_grade numeric(3, 2), -- e.g. 5.54
 tuition_paid boolean
);
```

Alter types after table creation

```
ALTER TABLE students
ALTER COLUMN name
TYPE varchar(128);
```

```
ALTER TABLE students
ALTER COLUMN average_grade
TYPE integer
-- Turns 5.54 into 6, not 5, before type conversion
USING ROUND(average_grade);
```

Let's apply this!

INTRODUCTION TO RELATIONAL DATABASES IN SQL

The not-null and unique constraints

INTRODUCTION TO RELATIONAL DATABASES IN SQL

Timo Grossenbacher
Data Journalist

datacamp

The not-null constraint

- Disallow `NULL` values in a certain column
- Must hold true for the current state
- Must hold true for any future state

What does NULL mean?

- unknown
- does not exist
- does not apply
- ...

What does NULL mean? An example

```
CREATE TABLE students (
 ssn integer not null,
 lastname varchar(64) not null,
 home_phone integer,
 office_phone integer
);
```

```
NULL != NULL
```

How to add or remove a not-null constraint

When creating a table...

```
CREATE TABLE students (
 ssn integer not null,
 lastname varchar(64) not null,
 home_phone integer,
 office_phone integer
);
```

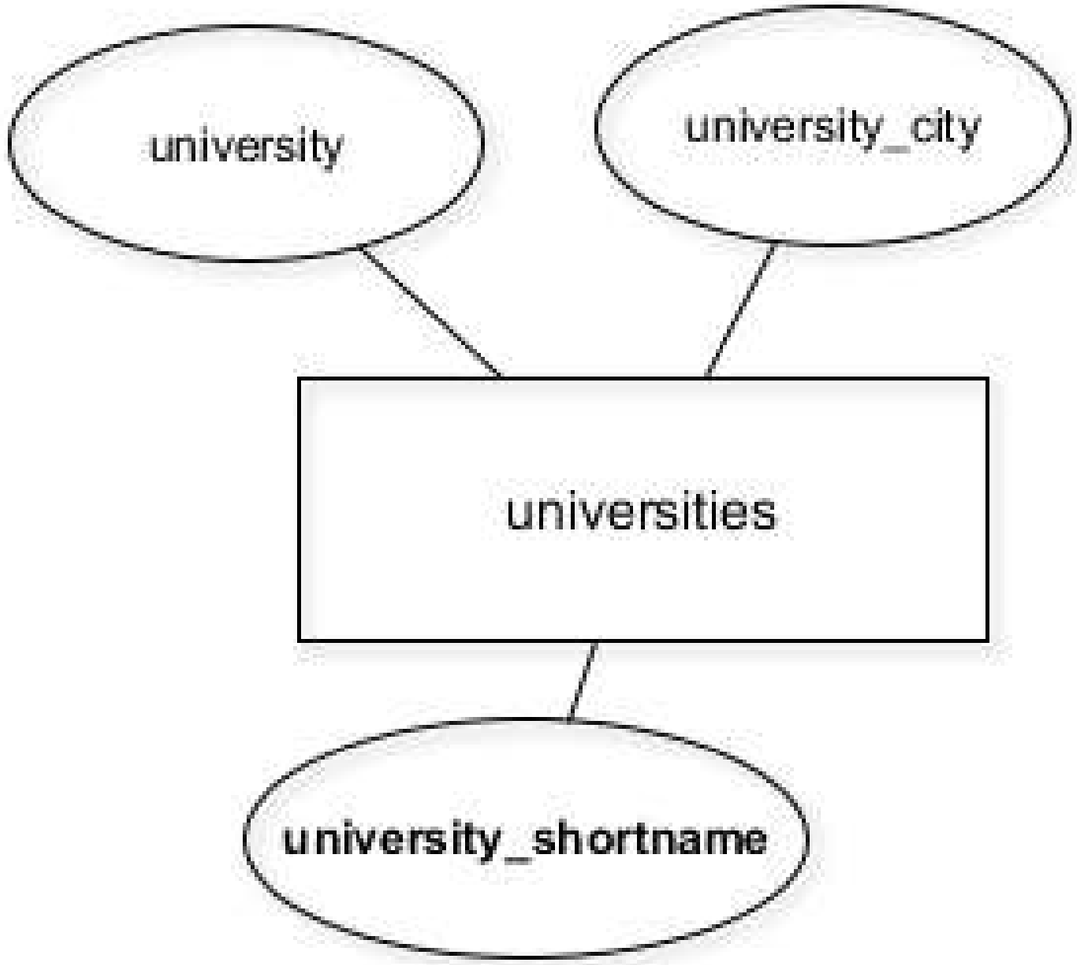
After the table has been created...

```
ALTER TABLE students
ALTER COLUMN home_phone
SET NOT NULL;
```

```
ALTER TABLE students
ALTER COLUMN ssn
DROP NOT NULL;
```

The unique constraint

- Disallow duplicate values in a column
- Must hold true for the current state
- Must hold true for any future state



Adding unique constraints

```
CREATE TABLE table_name (
 column_name UNIQUE
);
```

```
ALTER TABLE table_name
ADD CONSTRAINT some_name UNIQUE(column_name);
```

Let's apply this to the database!

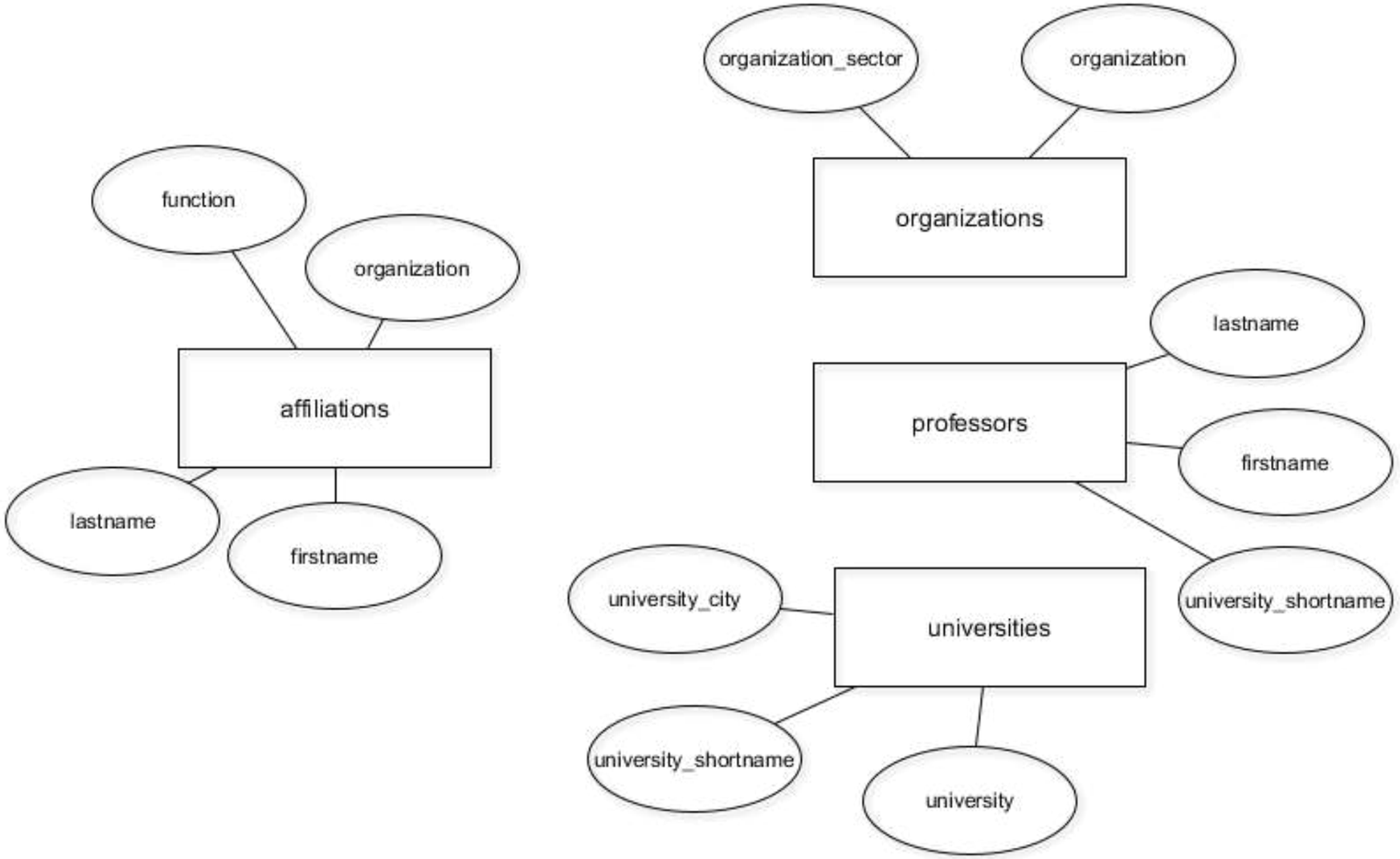
INTRODUCTION TO RELATIONAL DATABASES IN SQL

Keys and superkeys

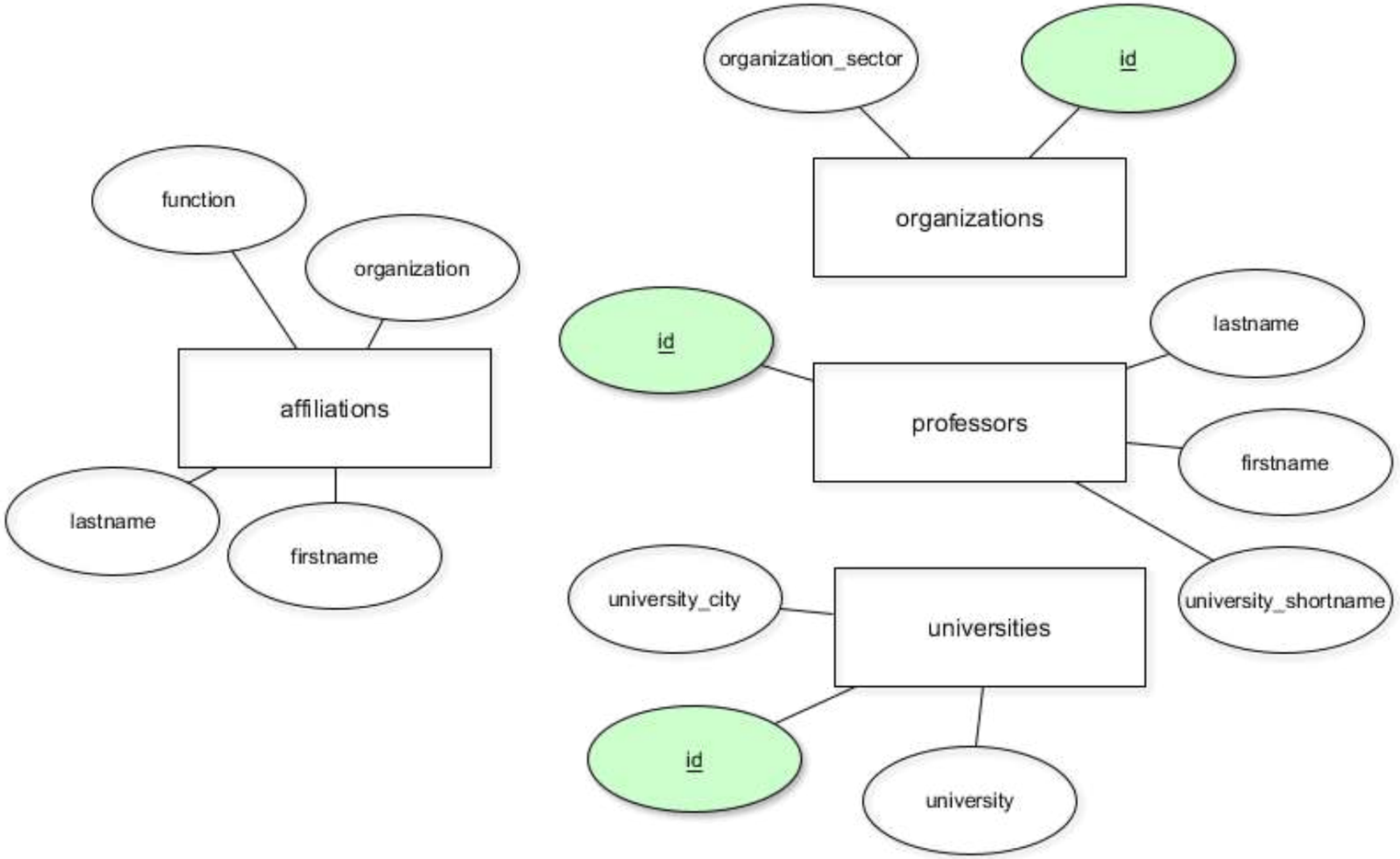
INTRODUCTION TO RELATIONAL DATABASES IN SQL

Timo Grossenbacher
Data Journalist

The current database model



The database model with primary keys



What is a key?

- Attribute(s) that identify a record uniquely
- As long as attributes can be removed: **superkey**
- If no more attributes can be removed: minimal superkey or **key**

license_no	serial_no	make	model	year
Texas ABC-739	A69352	Ford	Mustang	2
Florida TVP-347	B43696	Oldsmobile	Cutlass	5
New York MPO-22	X83554	Oldsmobile	Delta	1
California 432-TFY	C43742	Mercedes	190-D	99
California RSK-629	Y82935	Toyota	Camry	4
Texas RSK-629	U028365	Jaguar	XJS	4

SK1 = {license_no, serial_no, make, model, year}

SK2 = {license_no, serial_no, make, model}

SK3 = {make, model, year}, SK4 = {license_no, serial_no}, SKi, ..., SKn

Adapted from Elmasri, Navathe (2011): Fundamentals of Database Systems, 6th Ed., Pearson

license_no	serial_no	make	model	year
Texas ABC-739	A69352	Ford	Mustang	2
Florida TVP-347	B43696	Oldsmobile	Cutlass	5
New York MPO-22	X83554	Oldsmobile	Delta	1
California 432-TFY	C43742	Mercedes	190-D	99
California RSK-629	Y82935	Toyota	Camry	4
Texas RSK-629	U028365	Jaguar	XJS	4

K1 = {license_no}; K2 = {serial_no}; K3 = {model}; K4 = {make, year}

- K1 to 3 only consist of one attribute
- Removing either "make" or "year" from K4 would result in duplicates
- Only one candidate key can be the *chosen* key

Let's discover some keys!

INTRODUCTION TO RELATIONAL DATABASES IN SQL

Primary keys

INTRODUCTION TO RELATIONAL DATABASES IN SQL

Timo Grossenbacher
Data Journalist

datacamp

Primary keys

- One primary key per database table, chosen from candidate keys
- Uniquely identifies records, e.g. for referencing in other tables
- Unique and not-null constraints both apply
- Primary keys are time-invariant: choose columns wisely!

Specifying primary keys

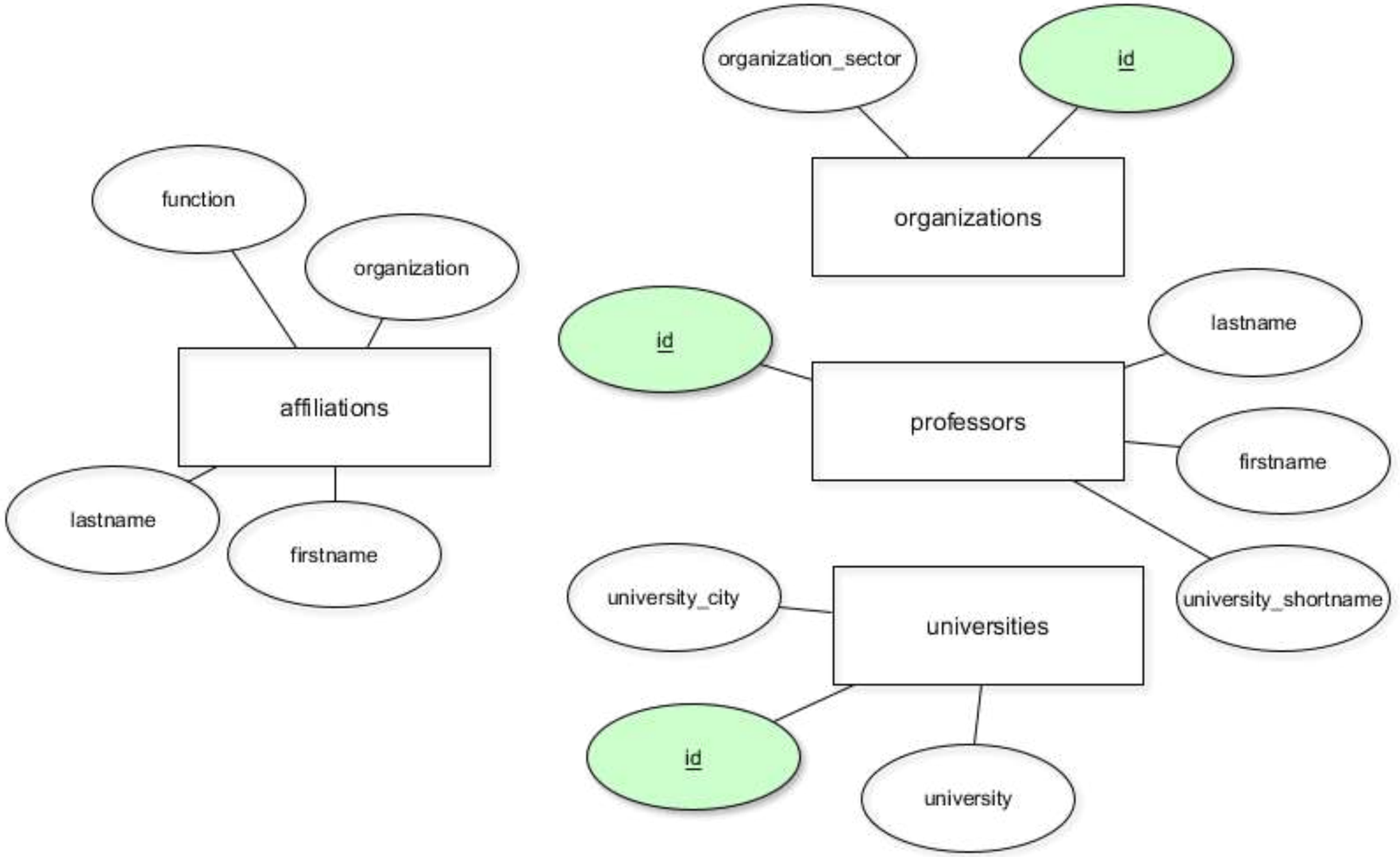
```
CREATE TABLE products (
    product_no integer UNIQUE NOT NULL,
    name text,
    price numeric
);
CREATE TABLE products (
    product_no integer PRIMARY KEY,
    name text,
    price numeric
);
```

```
CREATE TABLE example (
    a integer,
    b integer,
    c integer,
    PRIMARY KEY (a, c)
);
```

Taken from the PostgreSQL documentation.

Specifying primary keys (contd.)

```
ALTER TABLE table_name
ADD CONSTRAINT some_name PRIMARY KEY (column_name)
```


function
organization
affiliations
lastname
firstname
organization_sector
id
organizations
id
lastname
professors
firstname
university_city
universities
university_shortname
id
university

Let's practice!

INTRODUCTION TO RELATIONAL DATABASES IN SQL

Surrogate keys

INTRODUCTION TO RELATIONAL DATABASES IN SQL

Timo Grossenbacher
Data Journalist

Surrogate keys

- Primary keys should be built from as few columns as possible
- Primary keys should never change over time

license_no	serial_no	make	model	color
Texas ABC-739	A69352	Ford	Mustang	blue
Florida TVP-347	B43696	Oldsmobile	Cutlass	black
New York MPO-22	X83554	Oldsmobile	Delta	silver
California 432-TFY	C43742	Mercedes	190-D	champagne
California RSK-629	Y82935	Toyota	Camry	red
Texas RSK-629	U028365	Jaguar	XJS	blue

make	model	color
Ford	Mustang	blue
Oldsmobile	Cutlass	black
Oldsmobile	Delta	silver
Mercedes	190-D	champagne
Toyota	Camry	red
Jaguar	XJS	blue

Adding a surrogate key with serial data type

```
ALTER TABLE cars
ADD COLUMN id serial PRIMARY KEY;
INSERT INTO cars
VALUES ('Volkswagen', 'Blitz', 'black');
```

```
   make     |  model  | color       | id
 -----------+---------+-------------+-------------
 Ford       | Mustang | blue        | 1
 Oldsmobile | Cutlass | black       | 2
 Oldsmobile | Delta   | silver      | 3
 Mercedes   | 190-D   | champagne   | 4
 Toyota     | Camry   | red         | 5
 Jaguar     | XJS     | blue        | 6
 Volkswagen | Blitz   | black       | 7
```

Adding a surrogate key with serial data type (contd.)

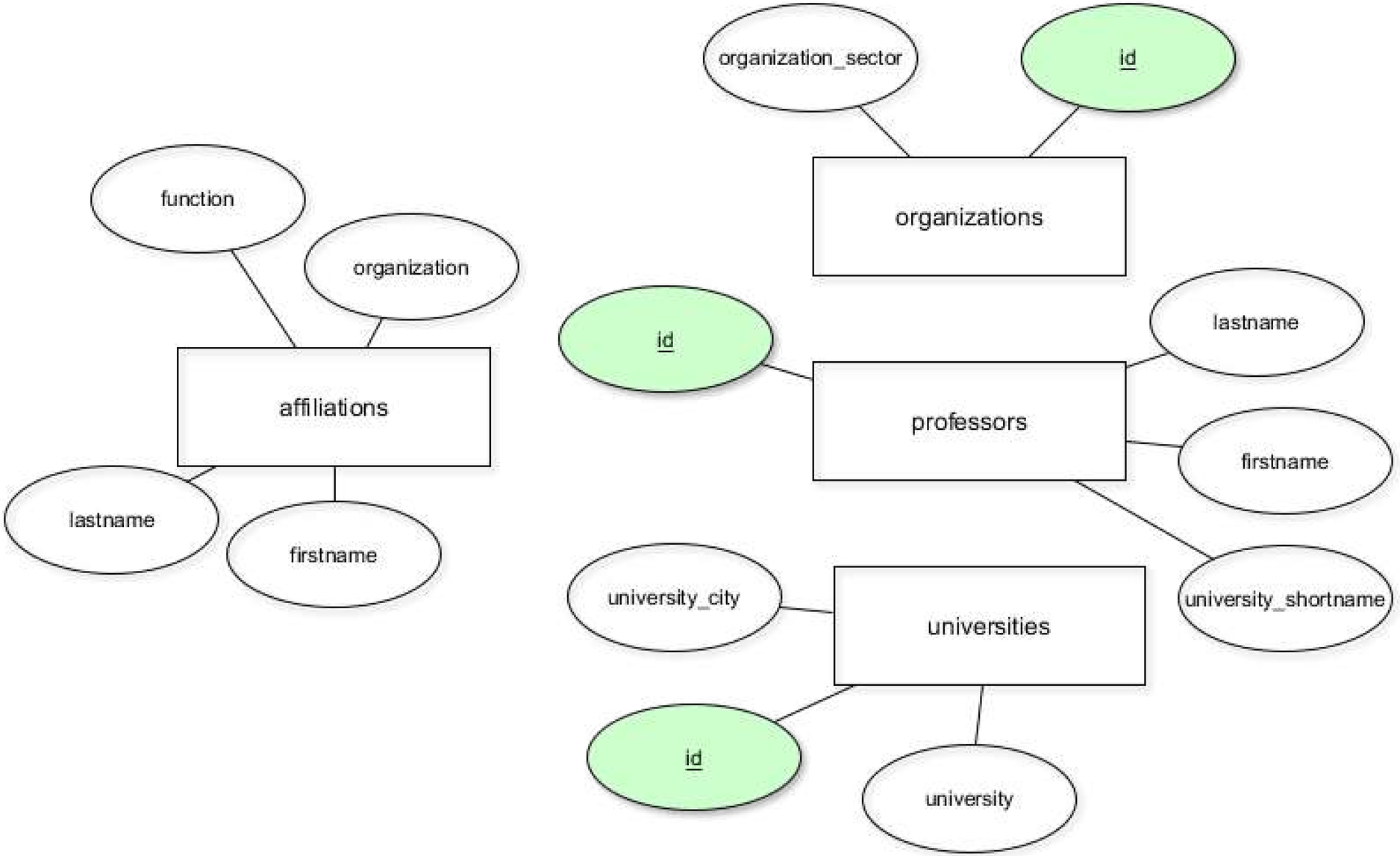
```
INSERT INTO cars
VALUES ('Opel', 'Astra', 'green', 1);
```

```
duplicate key value violates unique constraint "id_pkey"
DETAIL:  Key (id)=(1) already exists.
```

- "id" uniquely identifies records in the table – useful for referencing!

Another type of surrogate key

```
ALTER TABLE table_name
ADD COLUMN column_c varchar(256);

UPDATE table_name
SET column_c = CONCAT(column_a, column_b);
ALTER TABLE table_name
ADD CONSTRAINT pk PRIMARY KEY (column_c);
```


organization_sector
id
organizations
function
organization
affiliations
lastname
firstname
id
lastname
professors
firstname
university_city
universities
university_shortname
id
university

Let's try this!

INTRODUCTION TO RELATIONAL DATABASES IN SQL

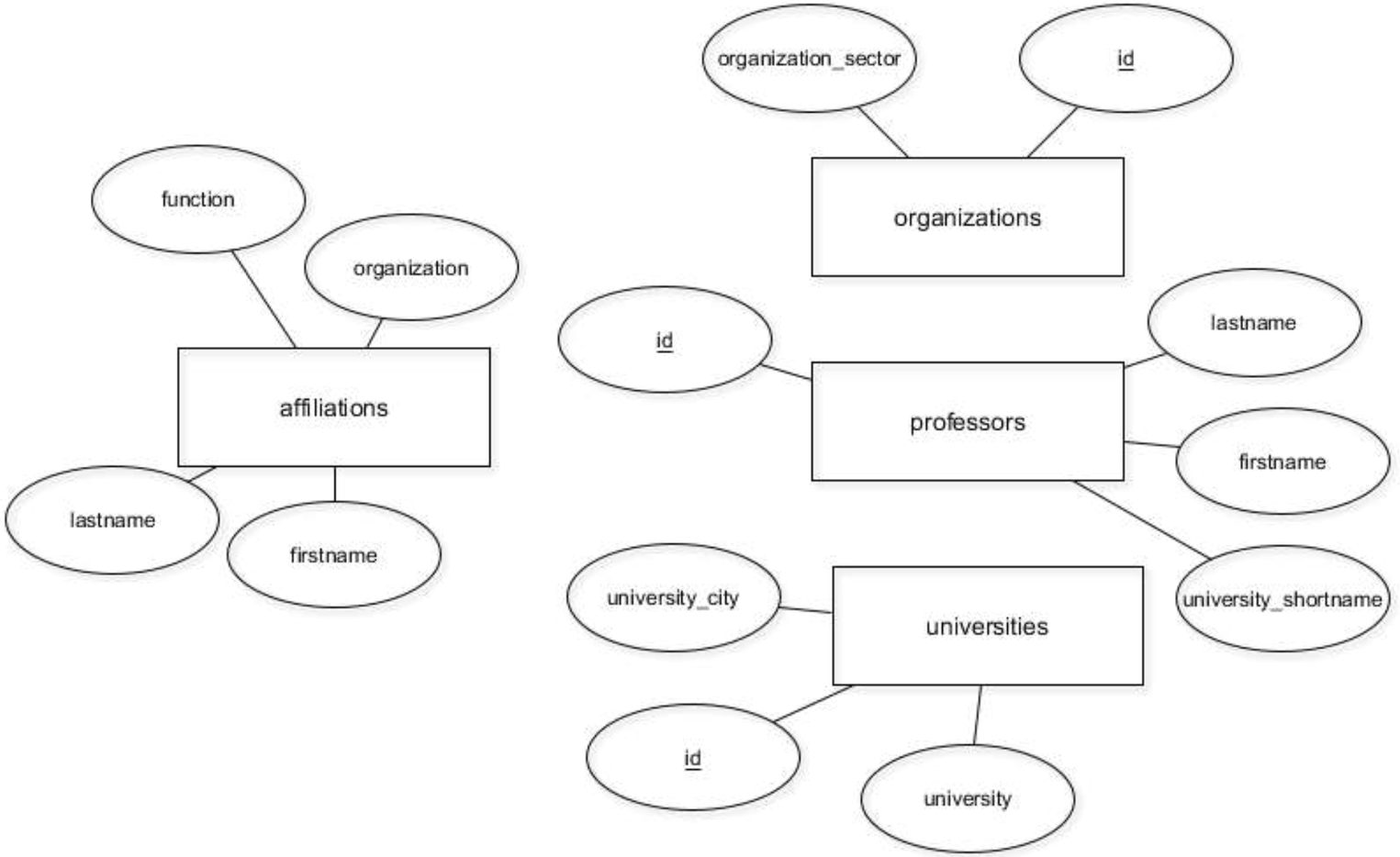
Model 1:N relationships with foreign keys

INTRODUCTION TO RELATIONAL DATABASES IN SQL

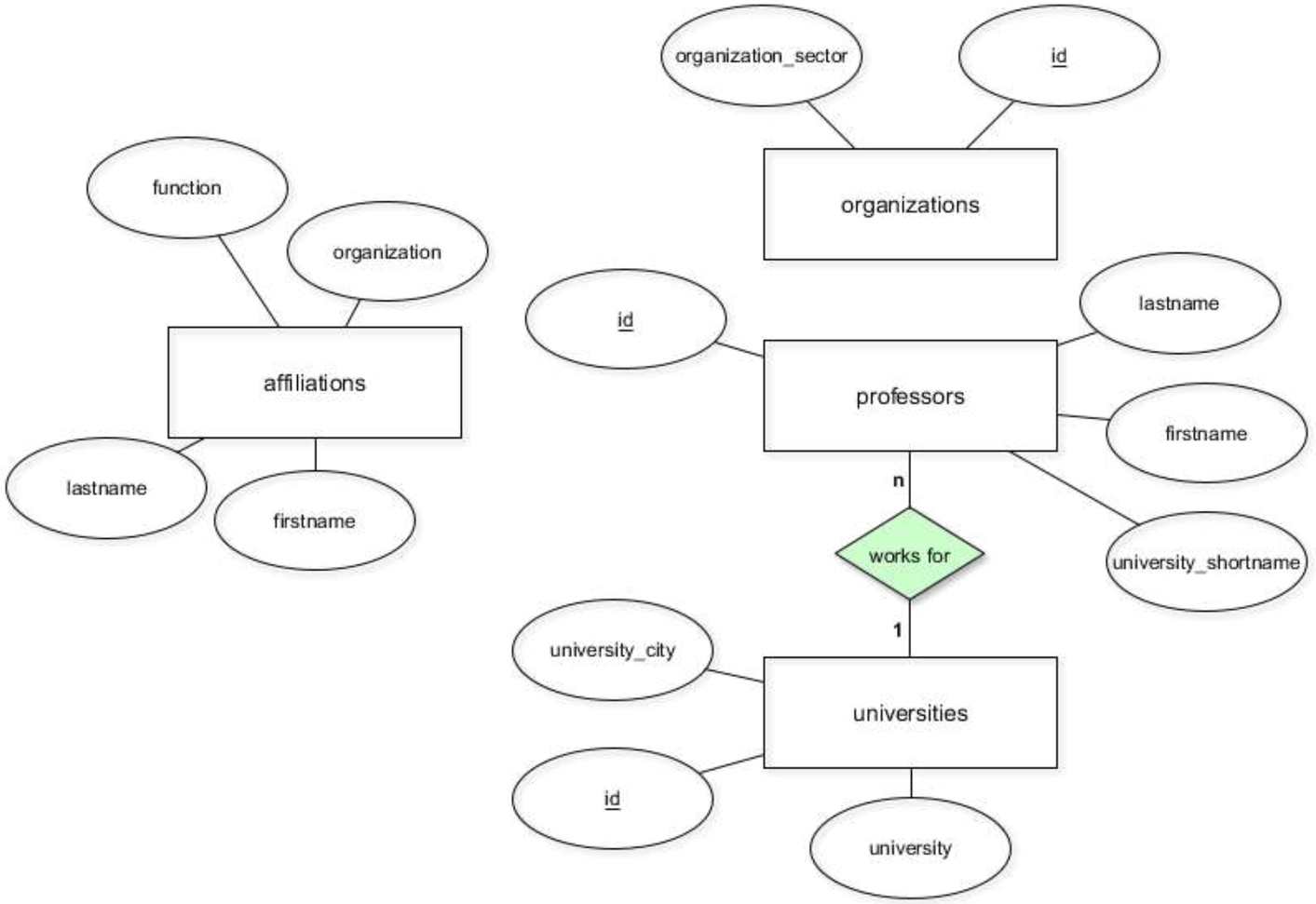
Timo Grossenbacher
Data Journalist

datacamp

The current database model



The next database model



Implementing relationships with foreign keys

- A foreign key (FK) points to the primary key (PK) of another table
- Domain of FK must be equal to domain of PK
- Each value of FK must exist in PK of the other table (FK constraint or "referential integrity")
- FKs are not actual *keys*

A query

```
SELECT * FROM professors LIMIT 8;
```

```
id |    firstname    |  lastname   | university_s..
 --+-----------------+-------------+------------
 1 | Karl            | Aberer      | EPF
 2 | Reza Shokrollah | Abhari      | ETH
 3 | Georges         | Abou Jaoudé | EPF
 4 | Hugues          | Abriel      | UBE
 5 | Daniel          | Aebersold   | UBE
 6 | Marcelo         | Aebi        | ULA
 7 | Christoph       | Aebi        | UBE
 8 | Patrick         | Aebischer   | EPF
```

```
SELECT * FROM universities;
```

```
 id  |   university   | university_city
<hr />--+----------------+----------------
 EPF | ETH Lausanne   | Lausanne
 ETH | ETH Zürich     | Zurich
 UBA | Uni Basel      | Basel
 UBE | Uni Bern       | Bern
 UFR | Uni Freiburg   | Fribourg
 UGE | Uni Genf       | Geneva
 ULA | Uni Lausanne   | Lausanne
 UNE | Uni Neuenburg  | Neuchâtel
 USG | Uni St. Gallen | Saint Gallen
 USI | USI Lugano     | Lugano
 UZH | Uni Zürich     | Zurich
```

Specifying foreign keys

```
CREATE TABLE manufacturers (
 name varchar(255) PRIMARY KEY);

INSERT INTO manufacturers
VALUES ('Ford'), ('VW'), ('GM');
CREATE TABLE cars (
 model varchar(255) PRIMARY KEY,
 manufacturer_name varchar(255) REFERENCES manufacturers (name));

INSERT INTO cars
VALUES ('Ranger', 'Ford'), ('Beetle', 'VW');
```

```
-- Throws an error!
INSERT INTO cars
VALUES ('Tundra', 'Toyota');
```

Specifying foreign keys to existing tables

```
ALTER TABLE a
ADD CONSTRAINT a_fkey FOREIGN KEY (b_id) REFERENCES b (id);
```

Let's implement this!

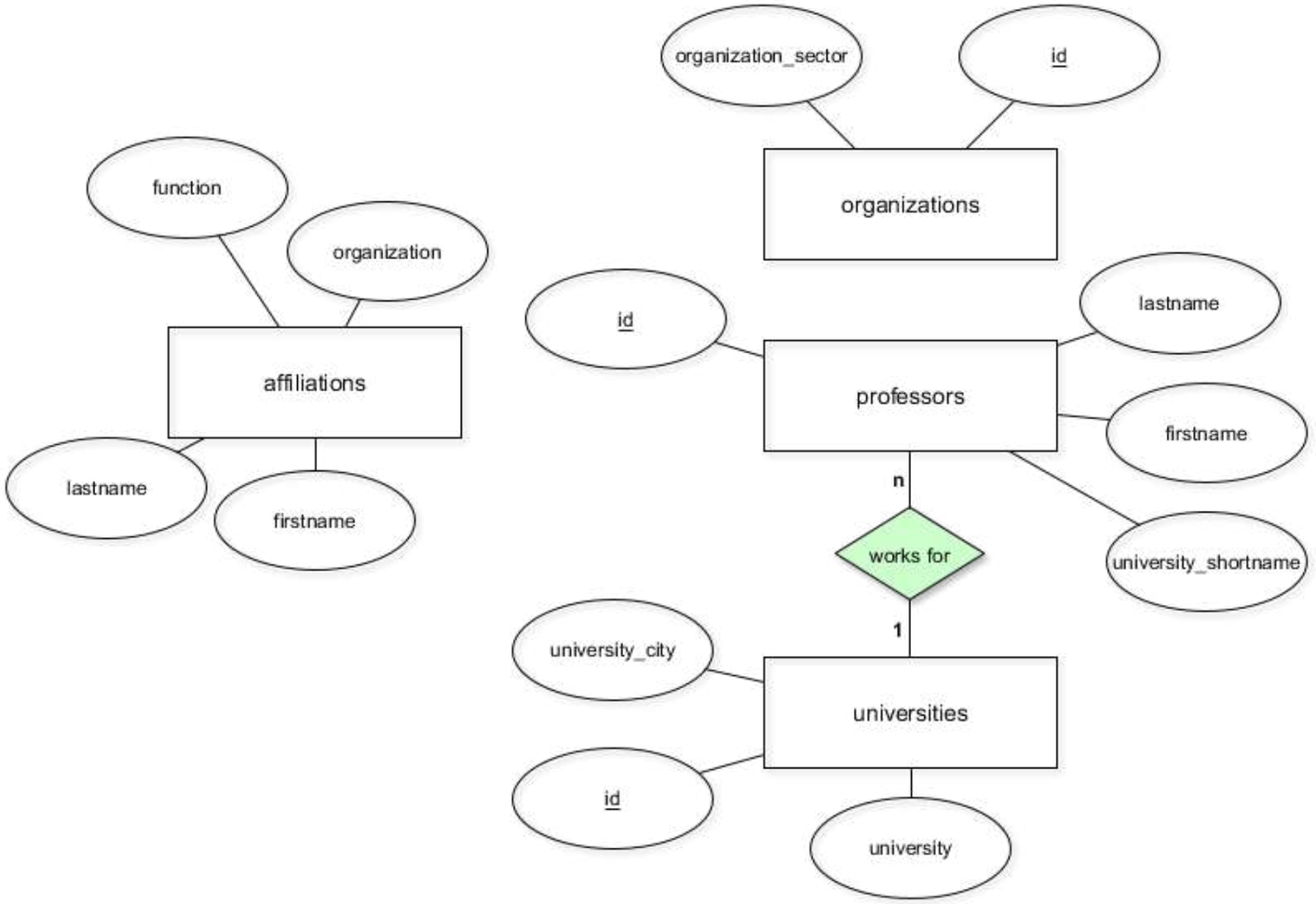
INTRODUCTION TO RELATIONAL DATABASES IN SQL

Model more complex relationships

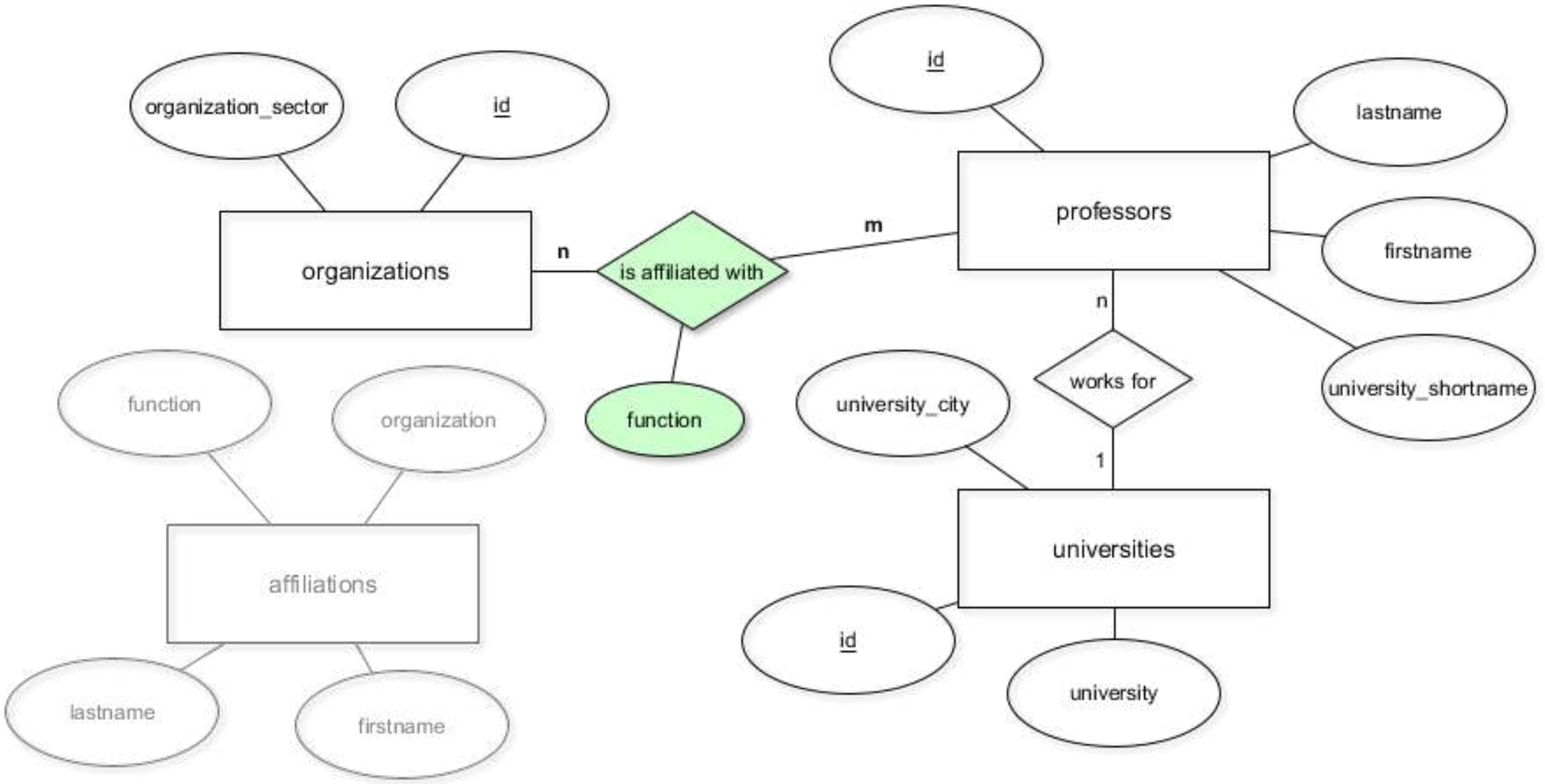
INTRODUCTION TO RELATIONAL DATABASES IN SQL

Timo Grossenbacher
Data Journalist

The current database model



The final database model



How to implement N:M-relationships

- Create a table
- Add foreign keys for every connected table
- Add additional attributes

```
CREATE TABLE affiliations (
 professor_id integer REFERENCES professors (id),
 organization_id varchar(256) REFERENCES organizations (id),
 function varchar(256)
);
```

- No primary key!
- Possible PK = {professor_id, organization_id, function}

Time to implement this!

INTRODUCTION TO RELATIONAL DATABASES IN SQL

Referential integrity

INTRODUCTION TO RELATIONAL DATABASES IN SQL

Timo Grossenbacher
Data Journalist

datacamp

Referential integrity

- *A record referencing another table must refer to an existing record in that table*
- Specified between two tables
- Enforced through foreign keys

Referential integrity violations

Referential integrity from table A to table B is violated...

- ...if a record in table B that is referenced from a record in table A is deleted.
- ...if a record in table A referencing a non-existing record from table B is inserted.
- Foreign keys prevent violations!

Dealing with violations

```
CREATE TABLE a (
 id integer PRIMARY KEY,
 column_a varchar(64),
 ...,
 b_id integer REFERENCES b (id) ON DELETE NO ACTION
);
```

```
CREATE TABLE a (
 id integer PRIMARY KEY,
 column_a varchar(64),
 ...,
 b_id integer REFERENCES b (id) ON DELETE CASCADE
);
```

Dealing with violations, contd.

ON DELETE...

- ...NO ACTION: Throw an error
- ...CASCADE: Delete all referencing records
- ...RESTRICT: Throw an error
- ...SET NULL: Set the referencing column to NULL
- ...SET DEFAULT: Set the referencing column to its default value

Let's look at some examples!

INTRODUCTION TO RELATIONAL DATABASES IN SQL

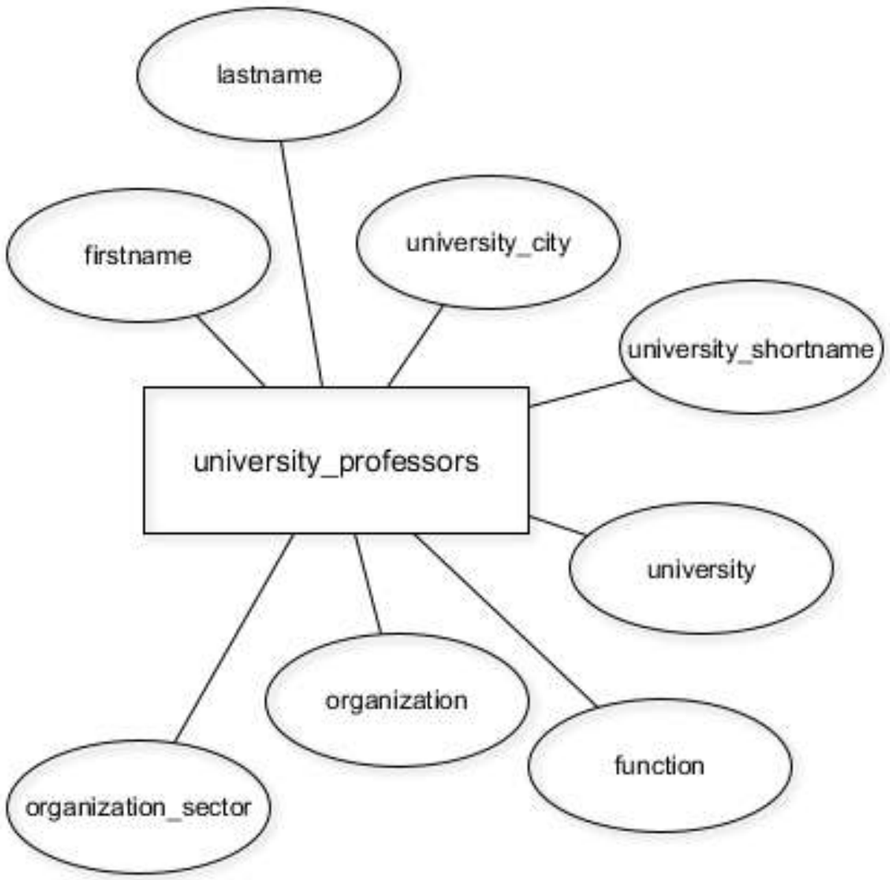
Roundup

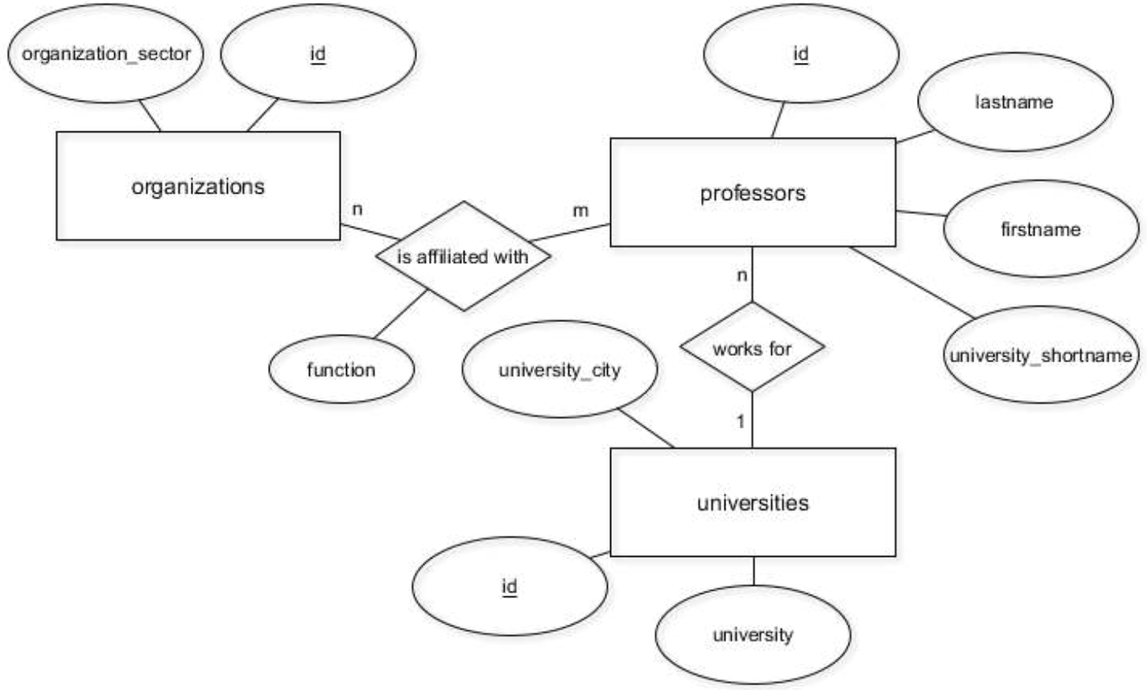
INTRODUCTION TO RELATIONAL DATABASES IN SQL

Timo Grossenbacher
Data Journalist

datacamp

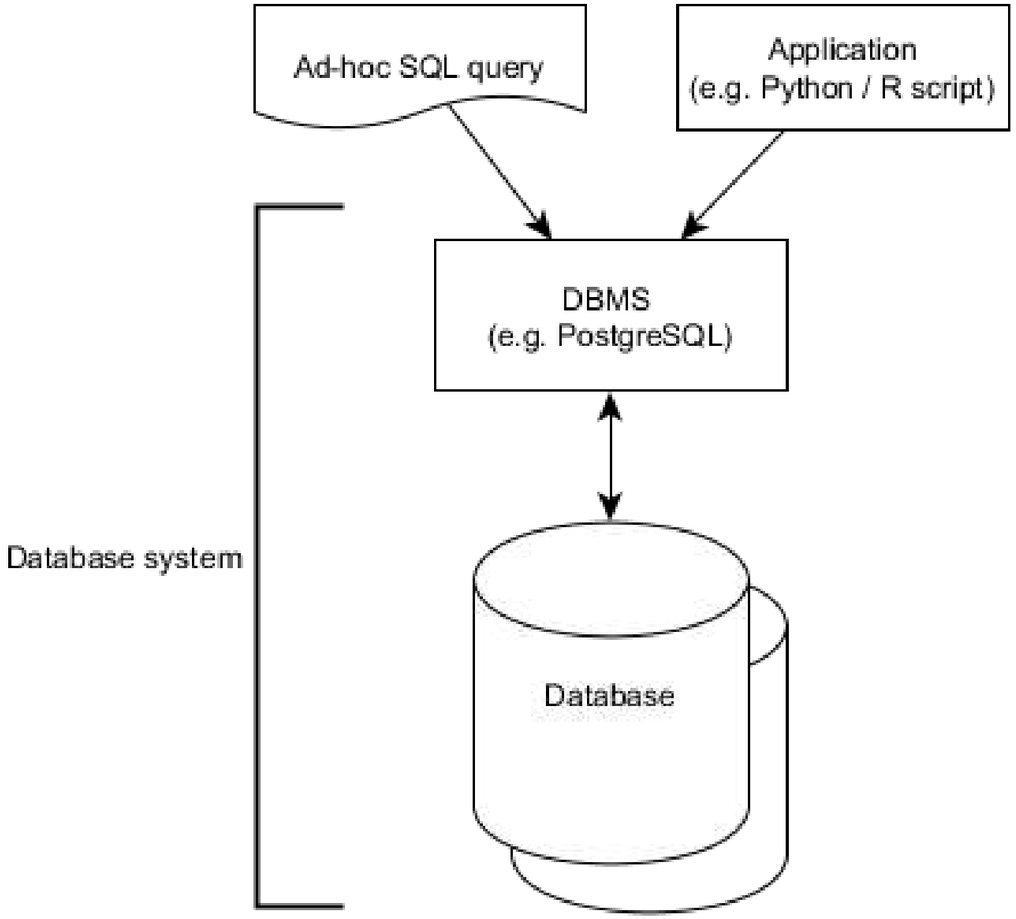
How you've transformed the database



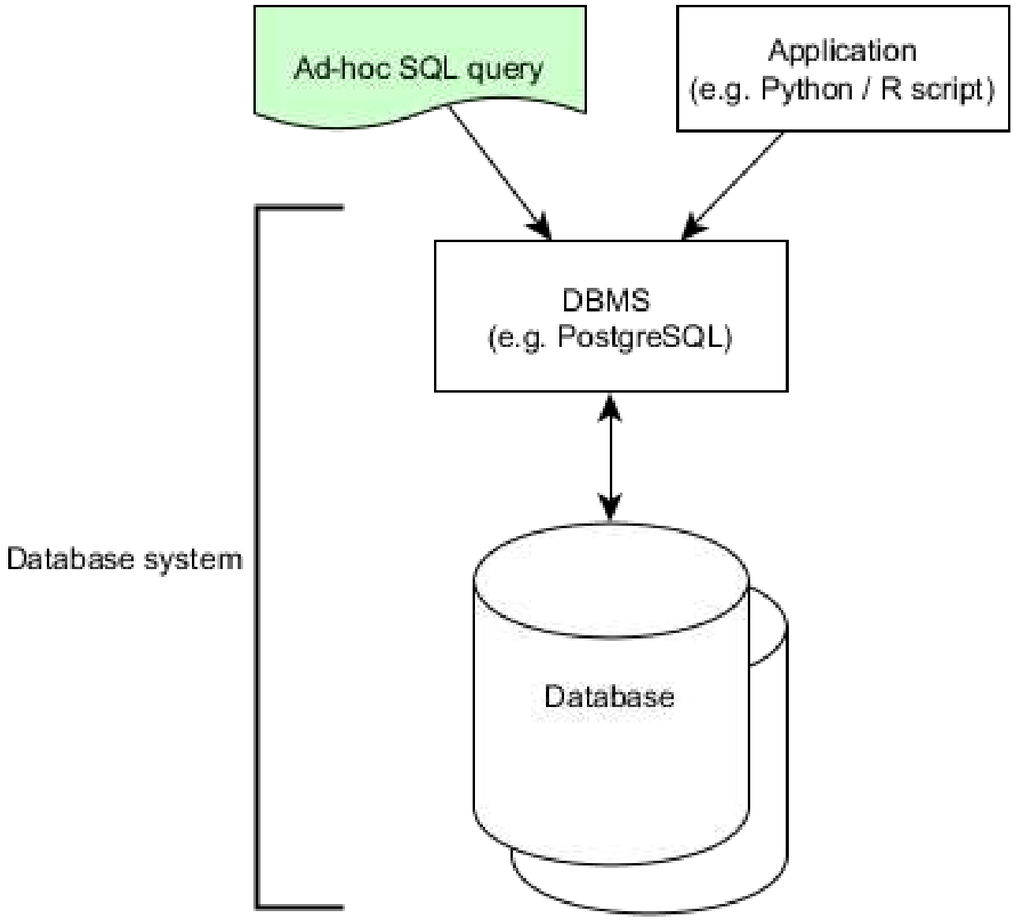


- Column data types
- Key constraints
- Relationships between tables

The database ecosystem



The database ecosystem



Thank you!

INTRODUCTION TO RELATIONAL DATABASES IN SQL